

DSA 4202 Assignment 2 Complete Workflow

This PDF is the full notebook-style workflow for the assignment. It documents data acquisition, exploratory analysis, cleaning, feature engineering, linear model development, tuning, evaluation, and final model justification.

1. Setup and Data Sources

```
import pandas as pd
import numpy as np

# Dataset 1: Medical Appointment No-Shows
no_show = pd.read_csv("noshowappointments-kaggle2-may-2016.csv")

# Dataset 2: Concrete Compressive Strength
concrete = pd.read_excel("Concrete_Data.xls")

# Models implemented:
# Ridge Regression, Lasso Regression, Logistic Regression, Linear SVC
```

Medical no-show data source: <https://raw.githubusercontent.com/swadeepsingh/Udacity-Data-Analyst-Nanodegree-NoShowAppointments/master/noshowappointments-kaggle2-may-2016.csv>

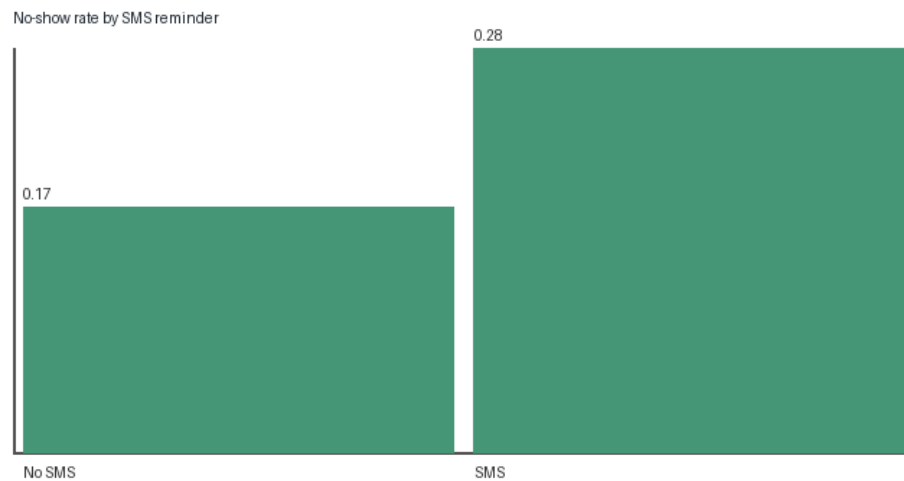
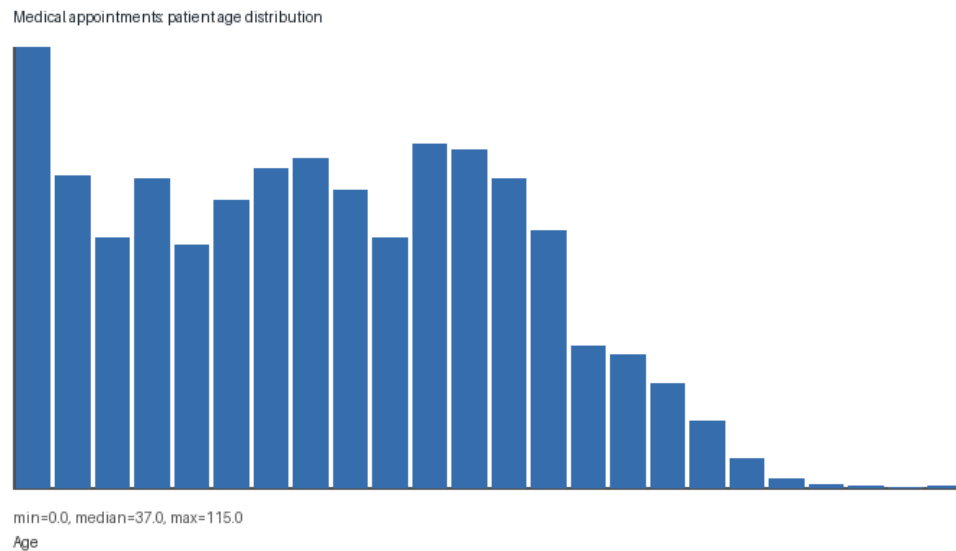
Concrete data source: <https://archive.ics.uci.edu/static/public/165/concrete+compressive+strength.zip>

2. Dataset 1 EDA: Medical Appointment No-Shows

Check	Result
Rows	110,527
Columns	14
Missing cells	0
Duplicate rows	0
Modeling rows after cleaning	110,521
Positive target rate	20.19%

```
no_show.info()
no_show.isna().sum()
no_show.describe(include="all")
no_show["No-show"].value_counts(normalize=True)
```

The raw dataset has no missing cells. Cleaning removed records with invalid age or negative waiting time. The classification target is imbalanced because most patients attended their appointment.



3. Dataset 1 Cleaning and Feature Engineering

```
work["scheduledday"] = pd.to_datetime(work["scheduledday"])
work["appointmentday"] = pd.to_datetime(work["appointmentday"])
work["waiting_days"] = appointmentday - scheduledday
work["appointment_dow"] = appointmentday.dayofweek
work["scheduled_hour"] = scheduledday.hour
work["target_no_show"] = (No-show == "Yes").astype(int)
work["gender_f"] = (Gender == "F").astype(int)

# Features:
# age, scholarship, hypertension, diabetes, alcoholism, handicap,
# sms_received, waiting_days, appointment_dow, scheduled_hour,
# gender_f, and top neighborhood indicators.
```

All features were standardized before fitting Logistic Regression and Linear SVC. Class weights were used so the minority no-show class was not ignored by the models.

4. Dataset 1 Modeling and Evaluation

```
# Logistic Regression
for lambda_value in [0.01, 0.08, 0.30]:
    fit weighted logistic regression with L2 penalty
    evaluate accuracy, precision, recall, and F1

# Linear SVC
for lambda_value in [0.001, 0.01, 0.05]:
    fit linear hinge-loss classifier with regularization
    evaluate accuracy, precision, recall, and F1
```

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.658	0.314	0.583	0.408
Linear SVC	0.653	0.306	0.565	0.397

Logistic regression confusion matrix

	Predicted 0	Predicted 1
Actual 0	TN 11950	FP 5691
Actual 1	FN 1863	TP 2600

Important feature	Coefficient
waiting_days	0.440
sms_received	0.164
age	-0.160
scheduled_hour	0.129
nbh_JARDIM CAMBURI	-0.090
scholarship	0.061
nbh_SANTA MARTHA	-0.055
nbh_ITARARÉ	0.055
nbh_JABOUR	-0.050
nbh_ILHA DO PRÍNCIPE	0.048

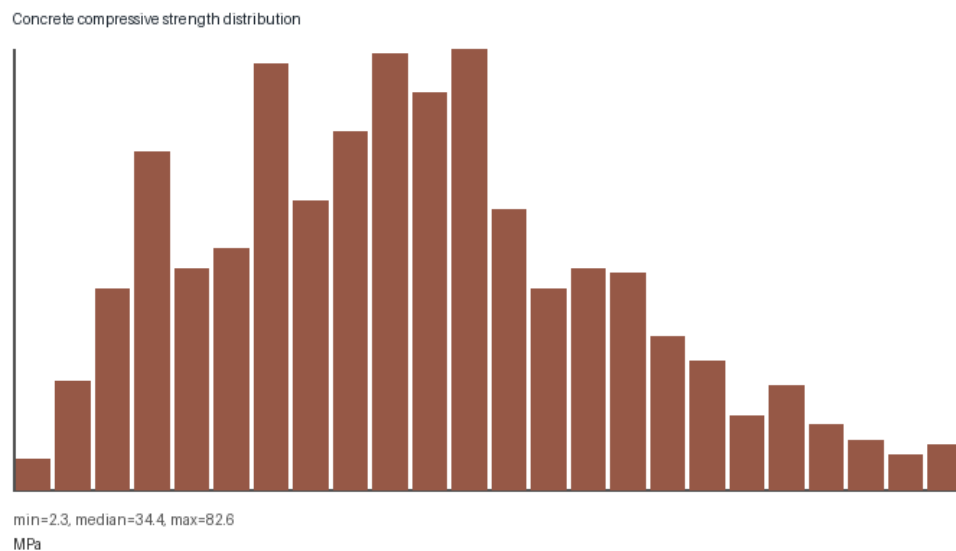
Logistic Regression was selected as the better classification model because it had the strongest F1 score in this workflow and gives interpretable probabilities for no-show risk.

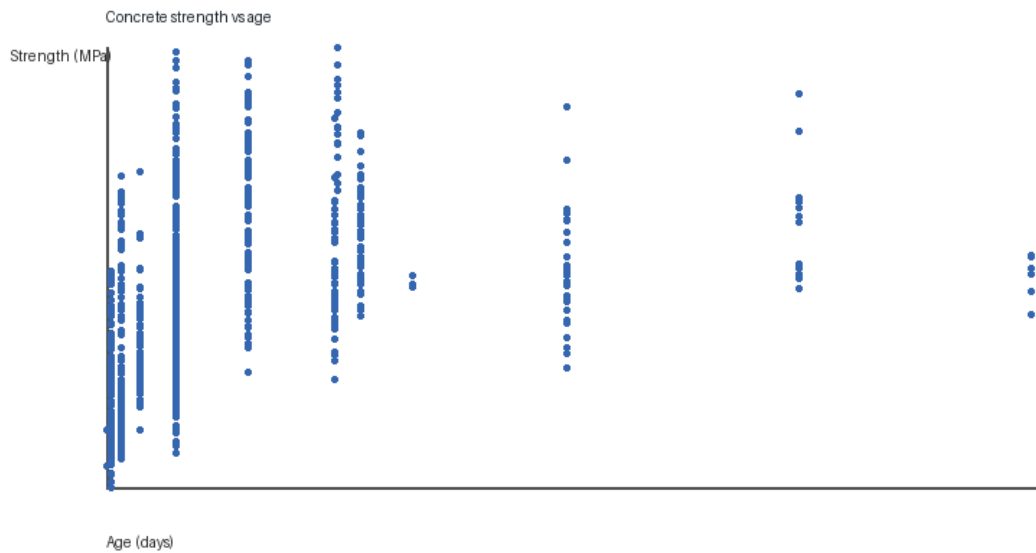
5. Dataset 2 EDA: Concrete Compressive Strength

Check	Result
Rows	1,030
Columns	9
Missing cells	0
Duplicate rows	25
Target	compressive_strength

```
concrete.info()  
concrete.isna().sum()  
concrete.describe()  
concrete.corr(numeric_only=True)["compressive_strength"].sort_values()
```

The concrete dataset is fully numeric. No missing cells were found. Strength increases with age but the relationship is nonlinear, which makes regularized linear models useful as interpretable baselines rather than perfect predictors.



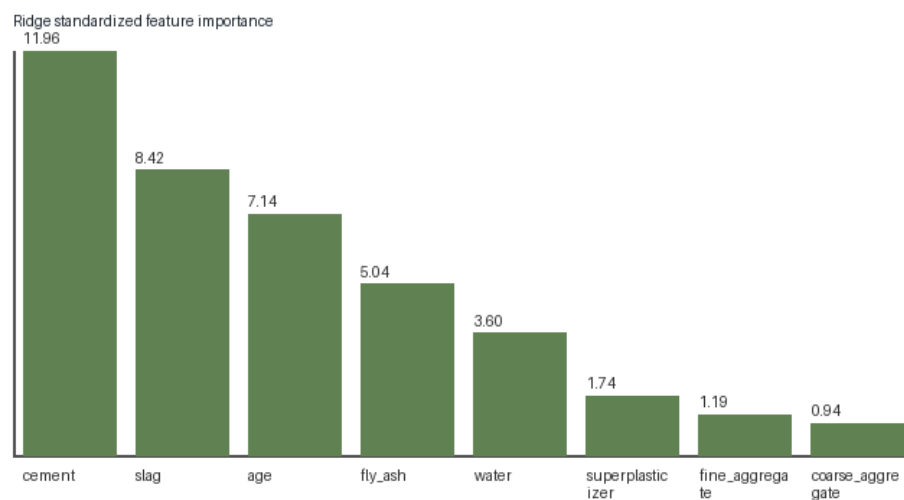


6. Dataset 2 Modeling and Evaluation

```
X = concrete.drop(columns=["compressive_strength"])
y = concrete["compressive_strength"]
X_train, X_test, y_train, y_test = train_test_split(X, y)
X_train_scaled, X_test_scaled = standardize(X_train, X_test)
```

```
# Ridge: minimize squared error + alpha * sum(beta^2)
# Lasso: minimize squared error + alpha * sum(abs(beta))
# Metrics: RMSE, MAE, and R2
```

Model	RMSE	MAE	R2
Ridge Regression	9.687	8.064	0.651
Lasso Regression	9.693	8.071	0.650



Feature	Standardized ridge coefficient
cement	11.965
slag	8.421
age	7.135
fly_ash	5.043
water	-3.599
superplasticizer	1.739
fine_aggregate	1.187
coarse_aggregate	0.936

7. Performance Improvement Techniques

Feature scaling was applied to both datasets because regularized linear models are sensitive to feature scale. Hyperparameter tuning was implemented through small grids over regularization strength. Class weighting was applied to the no-show classifier to improve recall for the minority class.

Best no-show Logistic Regression lambda: 0.01. Best Linear SVC lambda: 0.001. Best concrete Ridge alpha: 0.01. Best concrete Lasso alpha: 0.005.

8. Final Discussion

For the medical appointment classification task, Logistic Regression is recommended because it was competitive on F1 and provides clear coefficient-based interpretation. For the concrete regression task, Ridge Regression is recommended because it achieved the lowest RMSE while keeping all ingredient variables available for interpretation.

The main lesson is that linear methods are quick, transparent, and useful baselines, but preprocessing decisions matter. Scaling, class weighting, and regularization tuning had direct effects on model behavior and interpretability.